



OpenAster-1: Training an Open Language Model with 100B Tokens toward Long-Context and Multimodal Capabilities

Yaofeng Tu

Independent Researcher

yaofengtu28@gmail.com



github.com/binichallein/OpenAster-1



huggingface.co/collections/binichallein/openaster-1



modelscope.cn/collections/TYFTYF/OpenAster-1



Abstract

We present **OpenAster-1-2B**, a cost-conscious and fully open language model released with open data, training methodology, model weights, and experimental records. Such end-to-end openness remains uncommon in current model releases, where weights, data recipes, training logs, and implementation details are often only partially disclosed. OpenAster-1-2B adopts a Mixture-of-Experts architecture with approximately 2B total parameters and 1.3B activated parameters per token, and supports up to 128K context length through YaRN extrapolation. The model is trained on roughly 100B tokens through a staged pipeline consisting of pretraining and continued pretraining, yielding the base language model.

On top of the base model, we extend OpenAster-1-2B with visual understanding by adding a CLIP visual encoder and a projector module, obtaining OpenAster-VL, which reaches 1168.55 on the MME benchmark. Separately from the released model path, we also document a text-only post-training RL branch for mathematical reasoning: GSM8K-only GRPO, teacher-rewrite SFT, self-RFT, DPO, replay/KL preservation, and GPT-OSS teacher probes did not yet surpass the OpenAster-Math reference, motivating preservation-first objectives. OpenAster-1-2B is designed as a compact, low-cost, long-context, and multimodal open model recipe. By making the full path from data to deployable weights inspectable and reproducible, OpenAster aims to lower the barrier for independent reproduction and contribute to more open progress in the model community.

Keywords: large language models; long-context modeling; multimodal learning.

1. Introduction	2. Related Work
3. Model Architecture	4. Data Pipeline
5. Training Setup	6. Alignment & SFT
7. Post-Training RL Branch	8. Vision Instruction Tuning
9. Experiments	10. Conclusion
Appendix	References

1 Introduction

OpenAster-1-2B is built around a simple premise: a useful open model should expose not only weights, but also the data recipe, training path, evaluation records, and engineering failures that make the result reproducible. Current model releases often disclose only part of this chain. In contrast, OpenAster-1-2B is intended as a compact and cost-conscious 2B-parameter MoE model whose approximately 100B-token training budget, 128K-context extension, multimodal adaptation, and post-training studies can be inspected end to end.

The language model is first trained from scratch for 20B tokens and then continued for about 60B tokens on a rebuilt mixture to recover capability. After this base stage, we extend the context window from 4K to 32K, 64K, and 128K with YaRN-style rotary scaling. This produces the text foundation on top of which we attach additional capabilities rather than training a separate model for each capability setting.

To avoid overloading the paper with internal run identifiers, we use release-oriented model names: **OpenAster-128K-Base** for the final long-context base model, **OpenAster-Math** for the retained math-SFT text model, and **OpenAster-VL** for the fixed-resolution vision-instruction-tuned model. These names are used throughout the paper whenever a model variant, rather than an engineering run, is being discussed.

Our contributions are therefore fivefold. First, we provide a stage-by-stage account of a low-cost 2B MoE training run from scratch. Second, we describe a practical context-extension curriculum from 4K to 128K. Third, we report how OpenAster-1-2B is adapted into OpenAster-VL through a LLaVA-style visual tower and projector [25,26]. Fourth, we separately document a text-only post-training RL branch for

mathematical reasoning, including negative results from GRPO, teacher trace imitation, self-RFT, DPO, replay, KL preservation, and GPT-OSS probes. Fifth, we frame the release as an end-to-end open artifact: the goal is not only to publish a checkpoint, but to lower the barrier for independent reproduction and encourage more transparent progress in the model community.

2 Related Work

The closest line of work to OpenAster is not a particular architecture, but the small set of model releases that make the training process itself inspectable. OLMo is a leading example of this philosophy: the project explicitly frames openness as access to the full model flow, including pretraining data, post-training data, open training tools, evaluation tools, logs, and checkpoints rather than only final weights [1]. OLMo 2 further states that its released artifacts include model weights, full training data, training code and recipes, training logs, and thousands of intermediate checkpoints [2].

LLM360 makes the same point at the level of research norms. It argues that many open-source LLMs release only partial artifacts, and advocates releasing all training code and data, model checkpoints, and intermediate results so that the end-to-end training process can be transparent and reproducible [3]. Pythia is another important precedent: it releases models trained on public data in the same order, 154 checkpoints per model, and tools to reconstruct exact dataloaders, making learning dynamics and causal interventions on training possible [4].

BLOOM is an earlier open-access effort that demonstrated how a large collaborative project can publish a multilingual model together with unusually detailed training documentation [5]. OpenAster follows this transparency agenda at a smaller and lower-cost scale. Its contribution is to connect a 100B-token 2B MoE training run, long-context extension, multimodal adaptation, post-training results, data recipes, telemetry, and checkpoint paths into one release narrative, so that others can audit the decisions rather than only download the endpoint.

3 Model Architecture

OpenAster-1-2B is a Qwen3-style decoder-only MoE language model [22]. The retained configuration uses 18 transformer layers, hidden size 2048, FFN hidden size 6144 for dense components, 32 attention heads, 4 query groups, 64-dimensional KV channels, 4 routed experts, top-2 expert routing, and 3072-dimensional MoE expert FFNs. The tokenizer is the Qwen3.5-4B tokenizer used throughout the pretraining and long-context stages.

The initial maximum position length is 4096. During context extension, the model keeps the same core architecture but changes the positional scaling configuration: the 32K stage uses YaRN factor 8, and the 128K stage uses YaRN factor 32, both with original max position 4096 and rope base $1e6$. The 64K stage is a continuation between these endpoints. This follows the RoPE/YaRN long-context extension family [23,24].

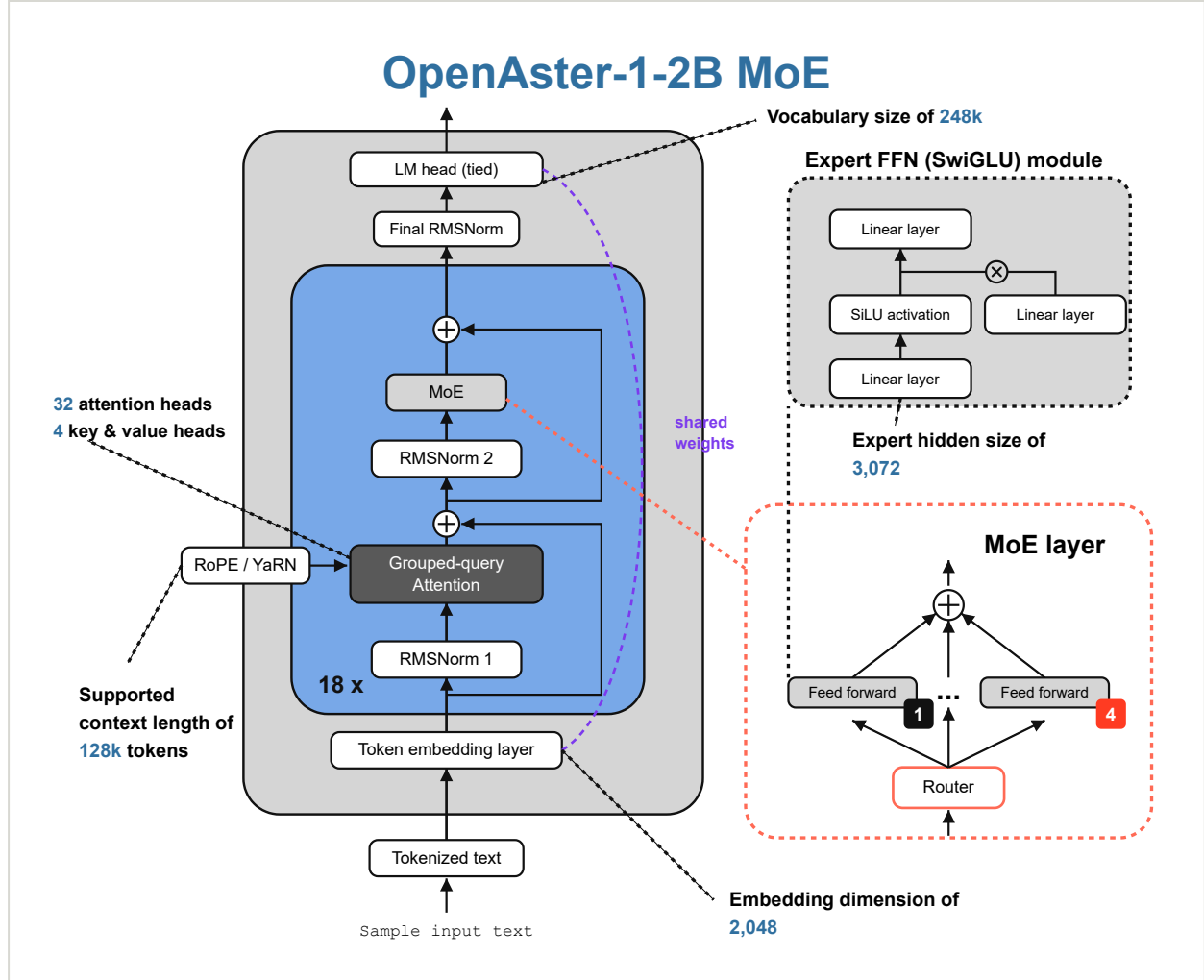


Figure 1: OpenAster-1-2B architecture. The decoder-only Qwen3-style MoE backbone follows the Qwen-family decoder pattern [20-22] and uses 18 layers, 2048 hidden dimensions, grouped-query attention with 32 attention heads and 4 key-value heads, and a routed MoE feed-forward block with 4 experts and top-2 routing. The input token embedding and LM head share weights. The retained 128K checkpoint uses a padded vocabulary size of 248,192 and YaRN/RoPE configuration with rope base 1e6 [23,24].

Table 1: Main architecture parameters.

Component	Value
Model family	Decoder-only Qwen3-style MoE language model
Layers / hidden size	18 layers / 2048 hidden
Attention	32 heads, 4 query groups, KV channel size 64
MoE	4 experts, top-2 routing, 3072 expert FFN hidden size
Tokenizer	Qwen3.5-4B tokenizer
Context schedule	4K pretraining -> 32K -> 64K -> 128K YaRN

We intentionally use a Qwen-like decoder architecture rather than introducing a new block design [20-22]. The goal of OpenAster is to make a low-cost open training path reproducible, so architectural novelty is less important than using a stable, well-understood recipe with strong tooling support. Qwen-style components such as RMSNorm, RoPE/YaRN-compatible positional encoding, grouped-query attention, SwiGLU feed-forward layers, and routed MoE layers are already supported by Megatron, Transformers, vLLM, and common conversion scripts. This reduces

implementation risk and allows the report to focus on data, training stability, checkpoint lineage, and release reproducibility.

We also keep the Qwen3.5-4B tokenizer throughout training instead of training a new tokenizer from scratch. Training a custom tokenizer is possible, but it would introduce another large source of variance: vocabulary construction, multilingual/code coverage, special-token conventions, chat-template compatibility, and downstream conversion behavior would all need separate validation. Reusing the Qwen tokenizer gives strong Chinese-English-code coverage, preserves mature special-token handling, and keeps the model compatible with existing Qwen-style tooling.

4 Data Pipeline

The data pipeline is staged. The 20B scratch stage uses DCLM Edu, OpenCSG Chinese FineWeb-Edu, FineWeb-Edu EN, FineMath 4+, Chinese FineWeb-Edu, and deduplicated Pile shards as a broad educational web, math, Chinese, and reference mixture. The 60B continued-pretraining stage rebuilds the mixture around replay from the 20B corpora, DCLM and FineWeb tails, FineMath and RedPajama ArXiv, OPC FineWeb code, The Stack, Codeforces, SkyPile-ZH, Pile reference tails, RedPajama C4/CommonCrawl, and StepFun Qwen3.5 chat [16]. The long-context stages progressively increase the maximum sequence length and shift toward math, filtered web, SFT-style reasoning, replay, and dialogue data [6-19].

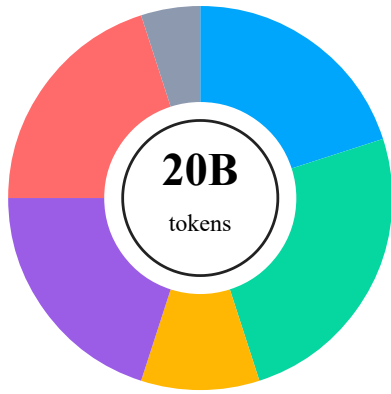
20B From-Scratch Mix

- DCLM Edu:** 4B tokens, 20.0%, English educational web.
- OpenCSG FineWeb Edu 4/5:** 5B tokens, 25.0%, Chinese educational web.
- FineMath 4+:** 2B tokens, 10.0%, English math corpus.
- FineWeb Edu EN:** 4B tokens, 20.0%, English educational web.
- Chinese FineWeb Edu:** 4B tokens, 20.0%, Chinese educational web.
- Pile Deduplicated:** 1B tokens, 5.0%, deduplicated general/reference replay.

60B Continued-Pretraining Mix

The 60B recipe allocates 12B tokens to old replay, 12B to web general, 11B to math/STEM, 10B to code, 6B to zh/multilingual, 6B to knowledge/reference, and 3B to StepFun chat [16]. The source-level budget includes replay DCLM/FineWeb/FineMath/Pile shards, DCLM and FineWeb tails, FineMath and RedPajama ArXiv, OPC FineWeb code, The Stack, Codeforces, SkyPile-ZH, Pile reference tails, RedPajama C4/CommonCrawl, and StepFun Qwen3.5 chat [16].

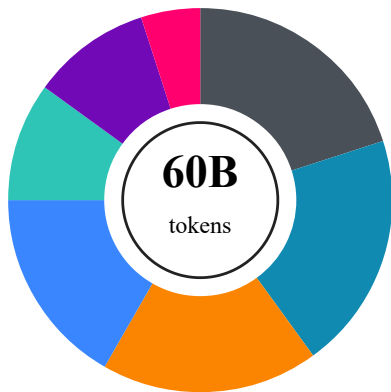
20B Scratch Mix	
	DCLM Edu: 4B tokens, 20.0%, English educational web.
	OpenCSG FineWeb Edu 4/5: 5B tokens, 25.0%, Chinese educational web.
	FineMath 4+: 2B tokens, 10.0%, English math corpus.
	FineWeb Edu EN: 4B tokens, 20.0%, English educational web.



Chinese FineWeb Edu: 4B tokens, 20.0%, Chinese educational web.

File Deduplicated: 1B tokens, 5.0%, general/reference replay.

60B Continued-Pretraining Mix



Old replay: 12B tokens, 20.0%, replay of the 20B mixture.

Web general: 12B tokens, 20.0%, DCLM/FineWeb and RedPajama web.

Math/STEM: 11B tokens, 18.3%, FineMath, arXiv, and STEM/reference data.

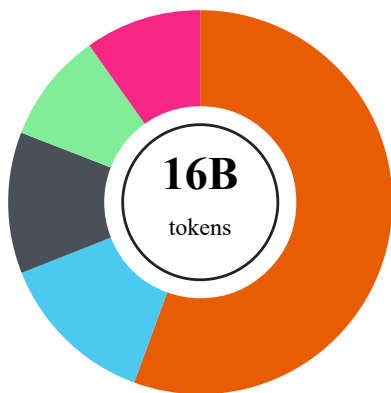
Code: 10B tokens, 16.7%, OPC FineWeb code, The Stack, and Codeforces.

Chinese / multilingual: 6B tokens, 10.0%, SkyPile-ZH and Chinese web tails.

Knowledge / reference: 6B tokens, 10.0%, Pile and RedPajama reference tails.

StepFun chat: 3B tokens, 5.0%, Qwen3.5 chat-style data.

Long-Context Extension Mix



Math L2/L3: 8.9B tokens, 55.6%, long-context math and reasoning data.

UltraFineWeb-L3: 2.15B tokens, 13.4%, filtered long-context web data.

Old replay: 1.9B tokens, 11.9%, replay from earlier stages.

UltraData-SFT: 1.48B tokens, 9.3%, SFT-style long-context reasoning data.

SFT / chat: 1.57B tokens, 9.8%, StepFun-SFT, UltraChat, and chat-style data.

Aggregate by High-Level Type

Chinese: 25.0B tokens, 26.0%, Chinese web, StepFun, and replay share.

English: 36.2B tokens, 37.7%, English educational/general web, reference, and chat data.

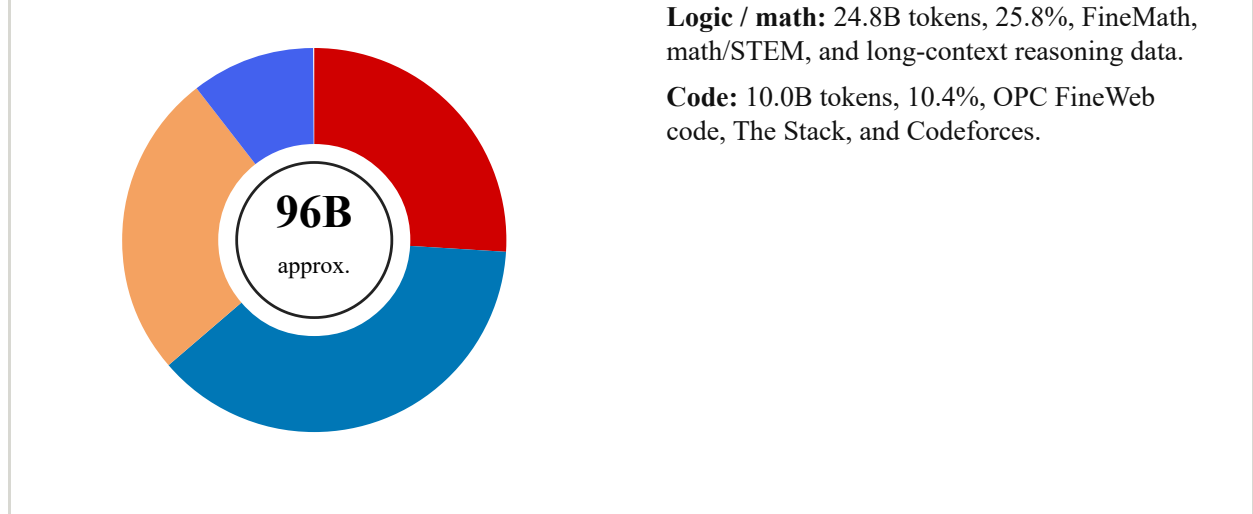


Figure 2: Data composition of the 20B from-scratch stage, the 60B continued-pretraining stage, the combined 32K/64K/128K long-context extension stages, and an approximate high-level category view over the staged text-training budget. Percentages are computed from stage token budgets; replay shards are assigned by their source-stage proportions. Colors are not recycled across different labels; Old replay keeps the same color where it appears across stages.

Data-Juicer cleaning applies unicode, email/IP/link cleanup, punctuation and whitespace normalization, length filtering from 64 to 200000 characters, special-character and repetition filters, exact document deduplication, and MinHash deduplication at Jaccard 0.82 [15].

Long-Context Mixes

The 32K stage uses 9B tokens: Math L2 5.0B, UltraData-SFT 1.283613B [17], old replay 1.0B, StepFun-SFT 0.716387B [16], UltraFineWeb-L3 0.5B [18], and UltraChat 0.5B [19]. The 64K stage uses 5B tokens: Math L3 2.6B, UltraFineWeb-L3 1.0B [18], old replay 0.7B, Math L2 0.5B, and SFT/chat 0.2B. The 128K stage uses 2B tokens: Math L3 0.8B, UltraFineWeb-L3 0.65B [18], old replay 0.2B, UltraData-SFT 0.2B [17], and UltraChat 0.15B [19].

The long-context pipeline uses `RAW_MIN_TOKENS=512`, token-length bucketing, stage-specific truncation caps, and exact-text sqlite deduplication against prior rendered corpora where applicable.

5 Training Setup

OpenAster is trained in six stages. The 20B stage is from scratch at sequence length 4096. The 60B stage continues from the 20B checkpoint at the same context length. The next three stages extend context length to 32K, 64K, and 128K, each continuing from the previous stage. The final stage is supervised fine-tuning from OpenAster-128K-Base to produce OpenAster-Math.

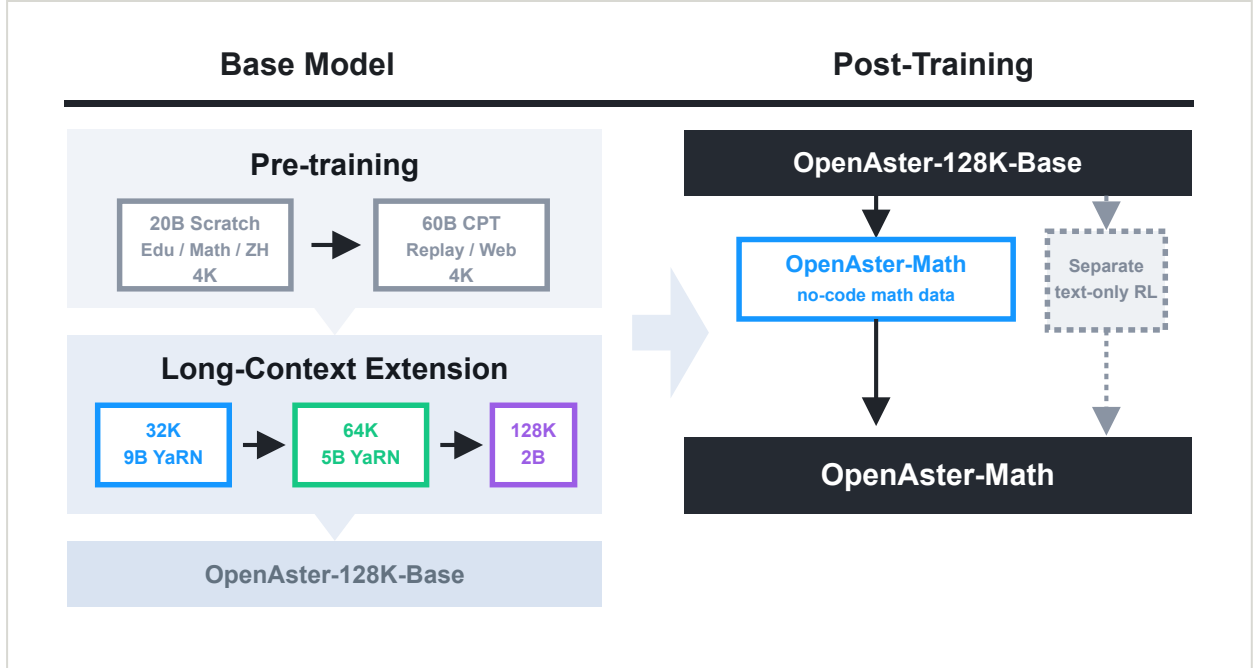


Figure 3: OpenAster base-model and post-training pipeline. The left side summarizes from-scratch pretraining, continued pretraining, and the 32K/64K/128K YaRN extension ladder; the right side shows the retained OpenAster-Math path and a separate text-only RL diagnostic branch.

Table 2: Core training-stage settings.

Stage	Seq / pos	Batching	LR schedule
20B scratch	4096 / 4096	micro 1, global 32	2e-4 -> 2e-5 cosine
60B CPT	4096 / 4096	micro 2, global 128	1e-4 -> 2e-5 cosine
32K YaRN	32768 / 32768	micro 1, global 64	5e-6 -> 5e-7 cosine
64K YaRN	65536 / 65536	micro 1, global 32	5e-6 -> 5e-7 cosine
128K YaRN	131072 / 131072	micro 1, global 16	3e-6 -> 3e-7 cosine

Table 2 reports the base-model training stages before OpenAster-Math. The `Seq / pos` column gives the packed training sequence length and the positional range used by RoPE/YaRN [23,24]. The `Batching` column reports the per-device micro batch size and the effective global batch size after data parallelism and gradient accumulation. The learning-rate column shows the initial and terminal learning rates under cosine decay.

The first two rows are 4K-context language-model training. The 20B row is the from-scratch run: the model is initialized randomly, trained with a relatively high 2e-4 learning rate, and kept at a smaller global batch of 32 while early optimization is still sensitive. The 60B continued-pretraining row starts from the 20B checkpoint, keeps the same 4K context window, increases the global batch to 128, and lowers the peak learning rate to 1e-4 so that the model can absorb a broader replay/web/math/ code mixture without overwriting the distribution learned in the scratch stage.

The last three rows are context-extension stages rather than new pretraining runs. The sequence length increases from 32K to 64K and then 128K, so the micro batch remains 1 and the global batch is reduced from 64 to 32 to 16 to fit memory and keep optimization stable. The 32K and 64K stages use a conservative 5e-6 -> 5e-7 schedule, while the 128K stage further lowers it to 3e-6 -> 3e-7 because the positional

extrapolation and per-sample token count are both largest at that point. The much smaller iteration counts in these rows reflect their shorter token budgets and much longer sequences. The OpenAster-Math setup is described separately in Section 6.

YaRN is the mechanism used for these context-extension rows. The 4K pretraining stages establish the model with a standard RoPE positional range; the YaRN stages then rescale the RoPE frequencies so that the same backbone can extrapolate to much longer positions without changing the transformer block structure. We do not jump directly from 4K to 128K. Instead, OpenAster climbs the context window through 32K, 64K, and 128K, each time continuing from the previous checkpoint and mixing long math, filtered web, replay, and SFT-style data. This staged schedule makes the positional extrapolation easier to monitor: each rung has its own retained checkpoint, validation loss/PPL measurements, and smaller learning rate than the 4K language-model training stages.

6 Alignment & SFT

OpenAster-Math is produced by supervised fine-tuning OpenAster-128K-Base on a no-code math instruction mixture. The run uses 136609 training examples and 2048 validation examples for one epoch. The optimizer performs 2135 update steps with micro batch size 1, gradient accumulation 8, world size 8, effective global batch size 64 examples/update, BF16, gradient checkpointing, learning rate $1e-5 \rightarrow 1e-6$, warmup ratio 0.03, and gradient clipping at 1.0.

The SFT data is math-only and no-code; the current report therefore treats OpenAster-Math as a math alignment stage rather than as a broad chat alignment or safety-tuning stage.

7 Post-Training RL Branch

We also maintain a separate text-only post-training branch for mathematical reasoning. This branch does not use the CLIP visual tower, image tokens, projector pretraining, or visual instruction data. Its starting point is an OpenAster-Math reference branch produced by additional math-reasoning SFT and teacher-rewrite preservation on top of OpenAster-128K-Base. The goal of this branch is diagnostic rather than release replacement: improve MATH500 above 10% and move GSM8K toward 20% while preserving the base policy's existing GSM8K correct set.

All key evaluations use MATH500 and GSM8K test with pass@1, temperature 0.7, top_p 0.95, n=1, max_model_len 16K, max_tokens 15360 where applicable, disable-thinking, and rule-based answer extraction. The historical OpenAster-Math reference is MATH500 $45/500 = 9.00\%$ and GSM8K $205/1319 = 15.54\%$, though same-root repeats show sampling variance such as MATH500 $49/500 = 9.80\%$ and GSM8K $166/1319 = 12.59\%$ or $187/1319 = 14.18\%$.

1. **OpenAster-Math reference.** This row is the reference policy produced by math-reasoning SFT with teacher-rewrite preservation from OpenAster-128K-Base. It sets the practical replacement threshold: a new branch should not be preferred unless it preserves both MATH500 and GSM8K.

2. **GSM8K-only GRPO.** This branch uses decontaminated GSM8K training data, $n=32$ rollouts, KL coefficient 0.005, and a small boxed-format reward. The best GSM8K score remains below the reference, and MATH500 is not the primary target of this branch.
3. **GLM-4-32B teacher rewrite SFT.** This branch uses 17,823 verified MATH12K/GSM8K teacher-rewrite examples under full SFT with a low learning rate. Despite the stronger teacher and verifier filtering, direct trace imitation degrades both held-out tasks.
4. **Self-RFT rank0 normalized SFT.** This branch uses 6,938 verified student-generated traces with style normalization. It briefly improves MATH500 beyond 10%, but GSM8K drops sharply, so the improvement is not a stable overall reasoning gain.
5. **DPO and teacher-hard DPO.** These branches use student chosen/rejected pairs or hard GLM-4-32B pairs with OpenAster-Math as the reference model. The preference objectives are trainable, but the best GSM8K result still remains below the reference.
6. **Replay / freeze / ASFT-KL.** These branches test base-output replay, replay-only controls, freeze probes, and ASFT/KL coefficient sweeps. They help diagnose preservation, but weak replay or weak KL is not sufficient to keep the base token distribution stable.
7. **GPT-OSS-120B teacher probes.** This branch uses verified GPT-OSS-120B traces with clean-final extraction, de-harmonized reasoning, and LoRA rank 8. The teacher is much stronger, but direct transfer still fails to produce stable gains on both MATH500 and GSM8K.

In Table 3, blue values mark the best score within each evaluation column.

Table 3: Text-only post-training RL branch summary.

Branch	Best MATH500	Best GSM8K
OpenAster-Math reference	9.00%	15.54%
GSM8K-only GRPO	Not primary	15.01%
GLM-4-32B teacher rewrite SFT	7.00%	10.39%
Self-RFT rank0 normalized SFT	10.20%	11.98%
DPO and teacher-hard DPO	Not primary	14.25%
Replay / freeze / ASFT-KL	8.00%	14.10%
GPT-OSS-120B teacher probes	10.60%	13.57%

Table 3 should be read relative to the OpenAster-Math reference: none of the explored branches produced a stable simultaneous gain on MATH500 and GSM8K, even when the teacher model itself was substantially stronger. The main lesson is that verified reasoning traces are not automatically good optimization targets for this 2B MoE student. Across GRPO, full SFT, DPO, replay, KL-style preservation, and LoRA probes, many updates changed the held-out correct set instead of adding stable new successes on top of the OpenAster-Math reference. Future work on this branch should therefore use a preservation-first objective: reference KL or distillation against the OpenAster-Math reference, teacher-as-ranker or teacher-as-repair rather than full-

trace imitation, and strict one-to-five checkpoint gates that stop immediately when GSM8K falls below the reference range. Given the limited pretraining budget and the still-weak base capability, our current judgment is that post-training alone is unlikely to substantially push OpenAster's reasoning ability beyond this range without further improving the base model.

8 Vision Instruction Tuning

For vision, OpenAster follows the LLaVA-style recipe [25,26]: starting from OpenAster-128K-Base, we add a CLIP ViT-L/14-336 visual tower, a two-layer GELU projector, image placeholder tokens, image-caption projector pretraining, and visual instruction tuning. The fixed-resolution LLaVA-1.5 path [26] establishes a working image-input baseline for OpenAster, reaching 1168.55 on MME under the same lmms-eval protocol used in our internal audit. This version supports static image inputs through a fixed 336-resolution visual pathway and keeps the multimodal extension deliberately conservative so that the effect of the visual tower, projector, and instruction-tuning data remains auditable.

Architecture

OpenAster follows a conservative LLaVA-style attachment [25,26]: a fixed-resolution CLIP visual tower produces image features, a two-layer GELU projector maps them into the language-model hidden space, and the resulting image tokens are concatenated with text tokens before entering the OpenAster-1-2B decoder.

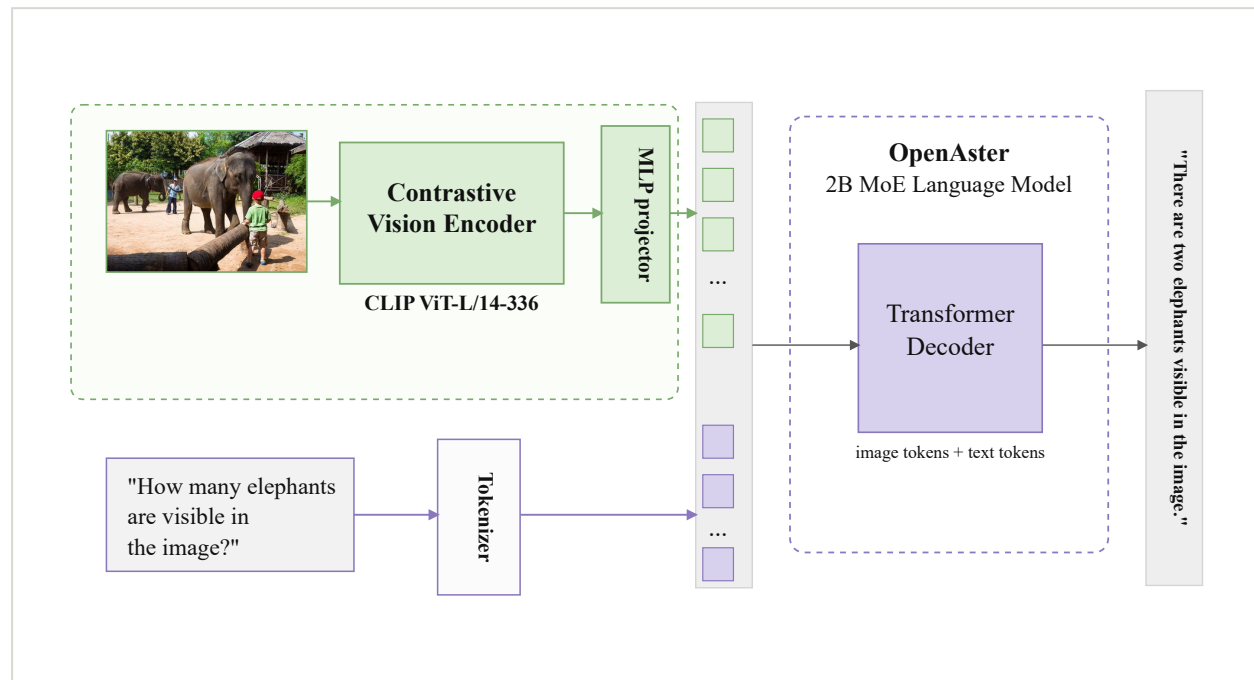


Figure 4: OpenAster-VL multimodal architecture, following the LLaVA family of vision-language attachment designs [25,26]. A CLIP ViT-L/14-336 visual tower encodes the image, a two-layer GELU projector maps visual features into the OpenAster hidden space, and the resulting image tokens are concatenated with tokenized text before entering the OpenAster-1-2B MoE decoder. The displayed QA is taken verbatim from the formal multimodal SFT data, COCO train2017/000000488541.jpg.

Training Recipe

OpenAster-VL was trained as an attachment to OpenAster-128K-Base rather than as a new model from scratch. Because the starting text model had not been broadly aligned as a chat model, the successful fixed-resolution run used a small text instruction warmup before visual training and kept a text replay stream during multimodal SFT. This differs from a clean two-stage LLaVA-1.5 reproduction [26], but it made the answer style and yes/no calibration more stable for this OpenAster model family.

The visual data sources are cited separately from the LLaVA method papers. Projector pretraining uses the LLaVA Visual Instruct Pretrain LCS-558K image-caption data [27]. Vision instruction tuning uses the LLaVA-1.5 `llava_v1_5_mix665k.json` mixture released under the LLaVA-Instruct-150K dataset repository [28].

Table 4: Vision instruction tuning recipe for OpenAster-VL.

Stage	Data	Trainable modules	Batch / LR	Steps	Role
Text warmup	51,974 Alpaca instruction examples	OpenAster-128K-Base LM	global 128 / 1e-5	407	Stabilize the text answer format before adding images.
Projector pretraining	558,128 LLaVA-Pretrain image-caption pairs [27]	Two-layer GELU projector	global 256 / 1e-3	2,181	Align CLIP visual features to the OpenAster hidden space.
Vision instruction tuning	665,296 LLaVA-1.5 visual instruction examples [28] + 4x text replay, 873,192 total	Projector + OpenAster-128K-Base LM + <code>lm_head</code>	global 64 / 2e-5	7,644	Teach image-conditioned instruction following while preserving text behavior.
Clean diagnostic rerun	558,128 pretrain pairs [27] and 665,296 LLaVA-1.5 SFT examples [28], no warmup or replay	Projector, then projector + OpenAster-128K-Base LM + <code>lm_head</code>	256 then 128 / 1e-3 then 2e-5	2,181 + 5,198	Isolate whether the same base can follow the standard LLaVA-1.5 recipe directly.

All fixed-resolution runs used a single static-image pathway with CLIP ViT-L/14-336 and `image_seq_len=576`. The CLIP tower was frozen during both projector pretraining and multimodal SFT. During SFT, image placeholders were rendered as `<|image_pad|>` tokens, text and image tokens were concatenated before the decoder, and the loss was applied only to assistant response tokens after the prompt mask.

Evaluation Results

OpenAster-VL establishes a working OpenAster image-input baseline. Under the internal MME script it obtains 1100.28 total score; under the default `lmms-eval` MME protocol used in the later audit, OpenAster-VL reaches 1168.55. The model remains below LLaVA-1.5-7B [26] on most vision benchmarks, especially OCR-heavy TextVQA and POPE hallucination splits, but it verifies that the CLIP-projector-OpenAster path is functional. The clean diagnostic rerun is intentionally reported as a negative control: without text warmup or replay, MME and MMBench drop sharply and the strict MME yes ratio rises to 90.02%, showing that answer-distribution

preservation is important for this model family. In Table 5, w/o warm denotes the standard LLaVA-1.5 two-stage recipe [26] without text warmup or replay, while w/ warm denotes the run with text warmup, projector pretraining, and 4x text replay during SFT. Blue values with upward arrows highlight the warmup run as the improved OpenAster-VL setting.

Table 5: OpenAster-VL vision benchmark results.

Model	MME total	POPE F1 random / popular / adv	ScienceQA IMG	TextVQA	MMBench-EN dev
OpenAster-VL w/o warm	891.59	71.61 / 69.63 / 70.22	41.10	25.39	42.84
OpenAster-VL w/ warm	1168.55 ↑	71.68 / 68.13 / 66.64 ↑	48.74 ↑	26.59 ↑	57.94 ↑
LLaVA-1.5-7B official [26]	1510.70	87.30 / 86.10 / 84.20	66.80	58.20	64.30

9 Experiments

The main text reports only basic run-health measurements: final validation loss/PPL by training stage, final SFT validation loss, and the headline 128K context-validation measurement. Detailed loss curves, gradient changes, channel validation loss, and full context-bucket PPL are in Appendix A.2-A.4.

Table 6: Minimal experimental summary in the main text.

Checkpoint	Primary measurement	Value	Interpretation
OpenAster-20B-Scratch	Mixed validation PPL	13.5905	Initial from-scratch model remained weak.
OpenAster-60B-CPT	Mixed validation PPL	9.4096	Capability-recovery continuation materially improved PPL.
OpenAster-32K	Default validation PPL	5.0628	Context-extension stage remained stable.
OpenAster-64K	Mixed validation PPL	4.6680	Run-health metric only; not same split as context buckets.
OpenAster-128K-Base	128K context-validation PPL	7.2110	Retained 128K bucket evaluation after context extension.
OpenAster-Math	Validation loss	0.8326	Validation loss flattened by step 1600-2100.

We additionally evaluate OpenAster-128K-Base before OpenAster-Math on four standard multiple-choice benchmarks. Because the evaluation server did not have external dataset access, we generated local lm-eval task mirrors from the official dataset rows and used vLLM loglikelihood scoring rather than free-form generation. MMLU, C-Eval, and C-MMLU use 5-shot evaluation; OpenBookQA uses the standard 0-shot setting.

Table 7: Basic knowledge evaluation of OpenAster-128K-Base.

Benchmark	Setting	Subjects	Samples	Macro Acc. (%)	Weighted Acc. (%)	Acc. Norm (%)
MMLU	5-shot	57	14,042	24.56	24.75	-
C-Eval valid	5-shot	52	1,346	24.08	24.07	24.07
C-MMLU	5-shot	67	11,582	25.21	25.12	25.12
OpenBookQA	0-shot	1	500	19.60	19.60	30.80

The evaluated model is OpenAster-128K-Base.

10 Conclusion

OpenAster-1-2B demonstrates a practical path for training a small fully open MoE model from scratch, recovering capability with continued pretraining, and extending the context window to 128K with a staged YaRN curriculum. The training record shows that the first 20B scratch model required substantial continued pretraining, that the long-context stages remained numerically stable, and that the OpenAster-Math stage produced a clean validation-loss plateau.

The separate text-only post-training branch adds a complementary finding: improving a fragile OpenAster-Math reference branch is not just a matter of adding more verified teacher or self-generated traces. For OpenAster, the next useful post-training direction is likely reference-preserved optimization that protects the base distribution while using teachers for ranking, repair, or small gated preference updates.

The next paper revision should add downstream task evaluations, safety benchmarks, release metadata, and contamination analysis. The current document establishes the training and telemetry foundation.

A Appendix

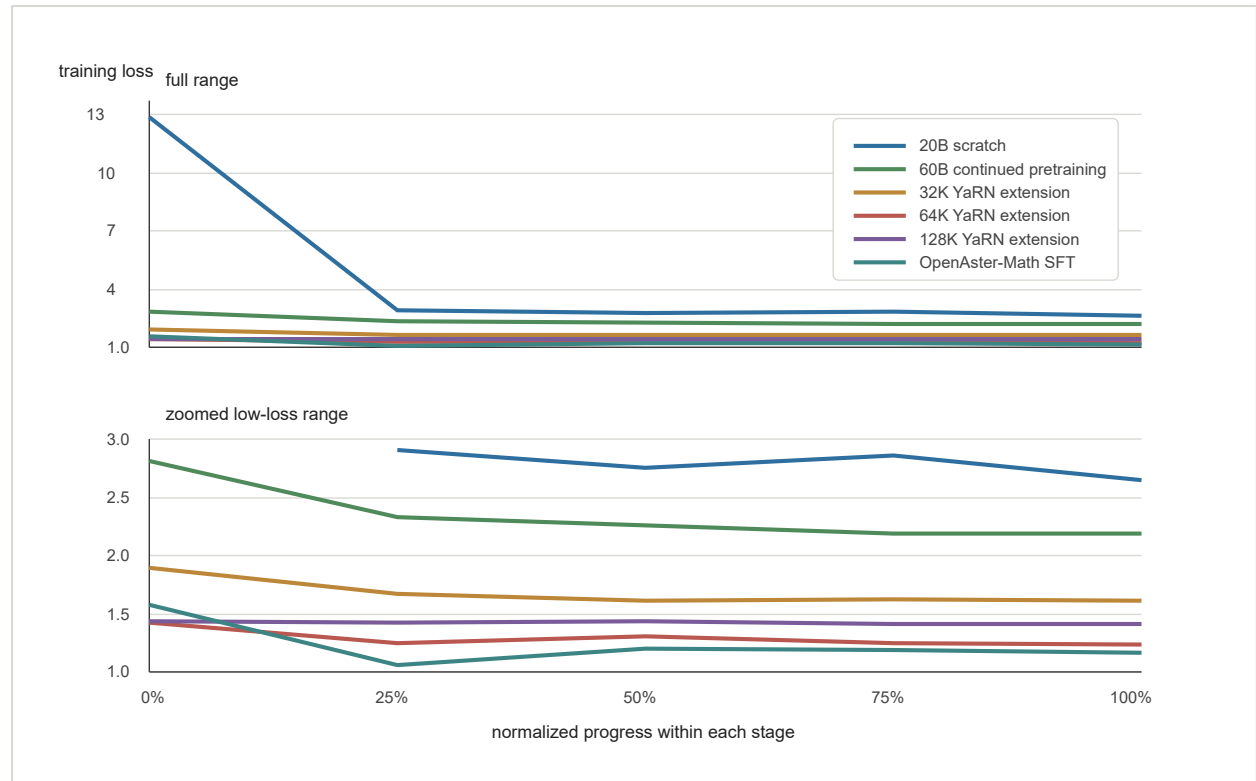
A.1 Detailed Hyperparameters

Appendix Table 1: Detailed training hyperparameters and checkpoint lineage.

Stage	Base checkpoint	Parallelism	Other controls
20B scratch	Random initialization; Qwen3.5-4B tokenizer	4 GPUs; TP1 PP1 EP2 CP1	WD 0.1, clip 1.0, seed 42, save/eval every 1B tokens
60B CPT	OpenAster-20B-Scratch at iter 152588	8 GPUs; TP1 PP1 EP1 CP1; DP8	WD 0.1, clip 1.0, BF16, fused attention, eval iters 20
32K YaRN	OpenAster-60B-CPT at iter 104825	8 GPUs; TP1 PP1 EP1 CP1	YaRN factor 8, original 4096, rope base 1e6, recompute MoE
64K YaRN	OpenAster-32K at iter 4293	8 GPUs; TP1 PP1 EP1 CP2; DP4	BF16/fused attention, selective recompute, clip 1.0
128K YaRN	OpenAster-64K at iter 2385	8 GPUs; TP1 PP1 EP1 CP4	YaRN factor 32, original 4096, rope base 1e6, recompute MoE

Stage	Base checkpoint	Parallelism	Other controls
OpenAster-Math	OpenAster-128K-Base	8 GPUs; DDP; BF16; gradient checkpointing	WD 0.0, clip 1.0, seed 1234, eval every 100 steps

A.2 Loss and Gradient Telemetry



Appendix Figure 1: Training loss trajectories sampled at approximately 0%, 25%, 50%, 75%, and 100% of each stage. The upper panel preserves the full loss range, while the lower panel zooms into the 1.0-3.0 loss band where the later-stage curves separate.

Appendix Table 2: Training loss, validation loss/PPL, and gradient summary.

Stage	Sampled train loss	Validation at end	Gradient summary
20B scratch	12.8729 -> 2.9117 -> 2.7588 -> 2.8620 -> 2.6435	loss 2.6094, PPL 13.5905	17.7053 -> 0.7728; max 112.6009
60B CPT	2.8068 -> 2.3351 -> 2.2587 -> 2.1941 -> 2.1844	loss 2.2417, PPL 9.4096	0.394 -> 0.451; max 7.977
32K YaRN	1.8884 -> 1.6694 -> 1.6143 -> 1.6223 -> 1.6132	default loss 1.6219, PPL 5.0628	2.257 -> 0.210
64K YaRN	1.4178 -> 1.2515 -> 1.3042 -> 1.2504 -> 1.2363	default loss 1.5683, PPL 4.7984	1.078 -> 0.180
128K YaRN	1.4397 -> 1.4262 -> 1.4398 -> 1.4131 -> 1.4131	default loss 1.0202, PPL 2.7739	0.742 -> 0.188
OpenAster-Math	1.5754 -> 1.0567 -> 1.1965 -> 1.1923 -> 1.1638	valid loss 0.8326	21.125 -> 1.4375; max 31.875

A.3 Channel Validation Loss

Appendix Table 3: Channel-validation loss and PPL at retained final checkpoints.

Channel validation data	60B loss	60B PPL	32K loss	32K PPL	128K loss	128K PPL
-------------------------	----------	---------	----------	---------	-----------	----------

Channel validation data	60B loss	60B PPL	32K loss	32K PPL	128K loss	128K PPL
old_replay_text_document	2.5507	12.816	2.5376	12.649	2.5447	12.739
knowledge_reference_text_document	2.7140	15.090	2.7170	15.134	2.7269	15.285
code_text_document	2.0635	7.873	2.0300	7.614	2.0304	7.617
math_stem_text_document	1.5770	4.840	1.5874	4.891	1.6042	4.974
stepfun_chat_text_document	1.3792	3.972	1.3505	3.859	1.3914	4.021
zh_multilingual_text_document	2.8358	17.043	2.8424	17.157	2.8630	17.515
web_general_text_document	2.6404	14.018	2.6366	13.965	2.6433	14.059

A.4 Context PPL

Appendix Table 4: Context-extension PPL measurements.

Checkpoint / protocol	4K	8K	16K	32K	64K	128K	Notes
32K YaRN context validation, iter 4293	9.512	9.219	9.097	8.751	-	-	Sequence-bucket validation retained for 4K-32K.
64K YaRN mixed train-log validation, iter 2385	-	-	-	-	4.668	-	Different validation/test split.
128K YaRN context validation, iter 954	9.712	9.411	9.366	8.974	8.700	7.211	Sequence-bucket validation retained for 4K-128K.

A.5 Qwen3-0.6B Post-Training Negative Control

We also ran a Qwen3-0.6B control to check whether a more post-trained text checkpoint is automatically better for LLaVA-style vision instruction tuning [25,26]. To keep the comparison aligned with Appendix Table 6, the reference row is the same Qwen3-0.6B-Base final checkpoint from the clean fixed-resolution LLaVA recipe [26]. The comparison row is the best strict-MME checkpoint from the Qwen3-0.6B post-trained branch trained with the same fixed-resolution recipe. Unlike OpenAster-VL, where text warmup and replay raise MME from 891.59 to 1168.55, the post-trained Qwen3-0.6B branch remains below the base reference. In Appendix Table 5, red values and downward arrows mark metrics that decrease relative to the base row.

Appendix Table 5: Qwen3-0.6B post-training negative-control results.

Model	Checkpoint condition	MME total	POPE avg F1	TextVQA	ScienceQA
Qwen3-0.6B-Base	Clean LLaVA final	1290.68	80.82	30.13	61.38
Qwen3-0.6B post-trained	Best strict-MME checkpoint	1241.76 ↓	80.19 ↓	29.93 ↓	59.30 ↓

The result suggests that text-side post-training does not automatically transfer into stronger multimodal alignment. For Qwen3-0.6B, the post-trained branch lowers MME and several general vision metrics under the same recipe. For OpenAster-VL, the warmup and replay mechanism is useful for a different reason: it stabilizes the answer distribution of a fragile MoE text backbone before multimodal SFT.

A.6 Qwen Dense-Base LLaVA-Style Scaling Sweep

Separately from the Qwen3-0.6B post-training negative control above, we trained several Qwen3 dense base models with the same LLaVA-1.5-style fixed-resolution recipe [26]: CLIP ViT-L/14-336, 576 image tokens, 558K projector pretraining samples [27], 665K LLaVA-1.5 visual instruction samples [28], no AnyRes, no ShareGPT4V, no MME calibration, and no extra text replay. Appendix Table 6 reports the final SFT checkpoint for each run. MME uses the strict-prompt total score; TextVQA uses the cleaned first-line short-answer score when that post-processing was required.

Appendix Table 6: Qwen dense-base models after LLaVA-style vision instruction tuning [25,26].

Base model	MME total	POPE F1 random / popular / adv	ScienceQA IMG	TextVQA	MMBench- EN dev
Qwen3-0.6B-Base	1290.68	84.08 / 81.48 / 76.91	61.38	30.13	68.17
Qwen3-1.7B-Base	1384.52	73.62 / 72.64 / 71.87	69.21	36.01	73.36
Qwen3-4B-Base	1528.23	80.41 / 79.44 / 78.10	74.27	41.03	78.25
Qwen3-8B-Base	1584.80	80.24 / 79.17 / 78.54	76.45	41.19	78.68

The main trend in Table 6 is clear: as the dense Qwen base scales from 0.6B to 8B, MME, ScienceQA, TextVQA, and MMBench generally rise under the same training recipe. POPE is less monotonic because it is sensitive to yes/no calibration, but the overall benchmark picture still improves with model scale. This result is useful mainly as a diagnostic contrast for OpenAster: the weakness of the OpenAster clean reproduction is not explained by the LLaVA recipe [25,26] alone, but by the interaction between the MoE base distribution, answer-format calibration, and the absence or presence of preservation-oriented text warmup/replay.

R References

1. Allen Institute for AI. [OLMo: Our fully open language model and complete model flow](#).
2. Team OLMo et al. [2 OLMo 2 Furious](#). arXiv:2501.00656, 2025.
3. Zhengzhong Liu et al. [LLM360: Towards Fully Transparent Open-Source LLMs](#). COLM, 2024.
4. Stella Biderman et al. [Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling](#). ICML, 2023.
5. BigScience Workshop. [BLOOM: A 176B-Parameter Open-Access Multilingual Language Model](#). arXiv:2211.05100, 2022.
6. MLFoundations. [DCLM-Baseline 1.0 dataset](#).
7. Hugging Face FineData. [FineWeb-Edu dataset](#); [The FineWeb Datasets technical report](#).
8. Hugging Face TB. [FineMath dataset](#).

9. OpenCSG. [Chinese FineWeb Edu dataset](#); Yijiong Yu et al., [OpenCSG Chinese Corpus](#), arXiv:2501.08197, 2025.
10. Leo Gao et al. [The Pile: An 800GB Dataset of Diverse Text for Language Modeling](#). arXiv:2101.00027, 2020.
11. Together Computer. [RedPajama-Data-1T dataset](#).
12. BigCode Project. [The Stack dataset](#).
13. Skywork. [SkyPile-150B dataset](#).
14. OpenCoder-LLM. [OPC FineWeb code corpus](#).
15. Data-Juicer. [Data-Juicer: data processing toolkit; A One-Stop Data Processing System for Large Language Models](#).
16. StepFun AI. [Step-3.5-Flash-SFT dataset](#).
17. OpenBMB / ModelBest. [UltraData-SFT-2605 dataset](#); [UltraData collection](#).
18. OpenBMB / ModelBest. [Ultra-FineWeb-L3 dataset](#); [Ultra-FineWeb dataset](#); [Efficient Data Filtering and Verification for High-Quality LLM Training](#).
19. Ning Ding et al. [Enhancing Chat Language Models by Scaling High-quality Instructional Conversations](#). arXiv:2305.14233, 2023; [UltraChat dataset](#).
20. Jinze Bai et al. [Qwen Technical Report](#). arXiv:2309.16609, 2023.
21. Qwen Team. [Qwen2.5 Technical Report](#). arXiv:2412.15115, 2024.
22. An Yang et al. [Qwen3 Technical Report](#). arXiv:2505.09388, 2025.
23. Jianlin Su et al. [RoFormer: Enhanced Transformer with Rotary Position Embedding](#). arXiv:2104.09864, 2021.
24. Bowen Peng et al. [YaRN: Efficient Context Window Extension of Large Language Models](#). arXiv:2309.00071, 2023.
25. Haotian Liu et al. [Visual Instruction Tuning](#). NeurIPS, 2023; arXiv:2304.08485.
26. Haotian Liu et al. [Improved Baselines with Visual Instruction Tuning](#). CVPR, 2024; arXiv:2310.03744.
27. Haotian Liu et al. [LLaVA Visual Instruct Pretrain LCS-558K dataset](#). Hugging Face dataset repository.
28. Haotian Liu et al. [LLaVA-1.5 llava_v1_5_mix665k visual instruction mixture](#), released under [LLaVA-Instruct-150K](#). Hugging Face dataset repository.